

Privacy-Preserving Machine Learning at Scale: From Training to Inference with FHE

Lux Partners
research@lux.network

January 2025

Abstract

We present Torus ML, a comprehensive framework for privacy-preserving machine learning using Fully Homomorphic Encryption, covering the entire ML lifecycle from training to deployment. Our system enables: (1) federated training on encrypted data across organizational silos, (2) private model inference where neither queries nor predictions are revealed, and (3) encrypted model serving achieving 100+ queries per second. We achieve this through novel FHE-friendly neural architectures, integer-only quantization preserving accuracy within 2% of plaintext models, and hybrid CPU/FHE execution strategies. Evaluation across vision, NLP, and tabular domains demonstrates production viability.

1 Introduction

Machine learning demands data—the more the better. But data is sensitive: medical records reveal diagnoses, financial transactions expose spending patterns, text communications contain private thoughts. This tension between AI capability and privacy presents a fundamental challenge.

Current approaches fall short:

- **Differential Privacy:** Adds noise, degrading model accuracy
- **Secure MPC:** High communication overhead, limited scalability
- **TEEs:** Trusted hardware assumptions with known vulnerabilities

- **Federated Learning:** Gradients leak substantial information [4]

Our Vision: FHE enables ML where data owners encrypt locally, models train and predict on ciphertext, and only authorized parties decrypt results.

1.1 Contributions

1. **FHE-Friendly Architectures:** Neural network designs optimized for homomorphic computation
2. **Quantization Framework:** Integer-only inference preserving accuracy
3. **Hybrid Execution:** Selective FHE layers minimizing overhead
4. **Encrypted Federated Learning:** Secure aggregation without trusted coordinator
5. **Production System:** 100+ QPS encrypted inference serving

2 Background

2.1 FHE for Machine Learning

Neural networks compute compositions of linear and non-linear functions:

$$f(\mathbf{x}) = \sigma_L(W_L \cdot \sigma_{L-1}(W_{L-1} \cdots \sigma_1(W_1 \cdot \mathbf{x} + b_1) \cdots + b_{L-1}) + b_L)$$

FHE natively supports linear operations. The challenge is non-linear activations.

Table 1: FHE Compatibility of ML Operations

Operation	FHE Support	Challenge	BatchNorm during inference reduces to affine transformation:
Addition	Native	None	
Matrix multiply	Native	Noise growth	
ReLU	Approximation	Non-polynomial	$y = \gamma \cdot \frac{x - \mu}{\sigma} + \beta$ (1)
Sigmoid/Tanh	Approximation	Range constraints	$= \underbrace{\frac{\gamma}{\sigma}}_s \cdot x + \underbrace{\left(\beta - \frac{\gamma\mu}{\sigma}\right)}_b$ (2)
Softmax	Difficult	Division, exponentiation	
BatchNorm	Inference-only	Pre-computed params	

2.2 TFHE Scheme

We use TFHE for its efficient bootstrapping:

Definition 2.1 (TFHE Ciphertext). A TFHE ciphertext of message $m \in \{0, 1\}$ is:

$$\text{ct} = (\mathbf{a}, b) \in \mathbb{T}^{n+1} \quad \text{where } b = \langle \mathbf{a}, \mathbf{s} \rangle + \frac{m}{2} + e$$

Bootstrapping refreshes noise in $O(1)$ gates:

$$\text{Bootstrap} : \text{ct} \mapsto \text{ct}' \quad \text{with } \text{noise}(\text{ct}') = O(1)$$

3 FHE-Friendly Neural Architectures

3.1 Polynomial Activation Functions

ReLU is not polynomial—problematic for FHE. We propose alternatives:

Definition 3.1 (Square Activation).

$$\sigma(x) = x^2$$

Naturally FHE-friendly, computes in one multiplication.

Definition 3.2 (Polynomial ReLU Approximation).

$$\text{ReLU}(x) \approx \frac{x + \sqrt{x^2 + \epsilon}}{2} \approx \frac{x + |x|}{2}$$

Approximated via polynomial expansion of $|x|$.

Definition 3.3 (SiLU Approximation).

$$\text{SiLU}(x) = x \cdot \sigma(x) \approx x \cdot (0.5 + 0.197x - 0.004x^3)$$

Cubic polynomial achieves $< 1\%$ error for $|x| < 5$.

3.2 FHE-Optimized Normalization

We pre-compute (s, b) after training—no FHE division needed.

3.3 Attention Without Softmax

Softmax is expensive: requires exponentiation and division. Alternatives:

Definition 3.4 (Linear Attention [3]).

$$\text{Attention}(Q, K, V) = \frac{\phi(Q)(\phi(K)^T V)}{\phi(Q)\phi(K)^T \mathbf{1}}$$

where ϕ is a feature map (e.g., $\phi(x) = \text{elu}(x) + 1$).

Definition 3.5 (Polynomial Attention).

$$\text{Softmax}(\mathbf{x})_i \approx \frac{\sum_{k=0}^d a_k x_i^k}{\sum_j \sum_{k=0}^d a_k x_j^k}$$

Degree-4 polynomial achieves $< 2\%$ approximation error.

4 Quantization for FHE

4.1 Integer-Only Inference

FHE operates on integers. We quantize all operations:

Definition 4.1 (Symmetric Quantization). For weights W with bit-width b :

$$s_w = \frac{\max(|W|)}{2^{b-1} - 1} \quad (3)$$

$$W_{\text{int}} = \text{round}\left(\frac{W}{s_w}\right) \quad (4)$$

Definition 4.2 (Asymmetric Quantization). For activations with range $[\alpha, \beta]$:

$$s_x = \frac{\beta - \alpha}{2^b - 1} \quad (5)$$

$$z = \text{round} \left(-\frac{\alpha}{s_x} \right) \quad (6)$$

$$x_{\text{int}} = \text{round} \left(\frac{x}{s_x} \right) + z \quad (7)$$

4.2 Quantized FHE Operations

Algorithm 1 Quantized FHE Linear Layer

Require: Encrypted input $\text{Enc}(x_{\text{int}})$, quantized weights W_{int} , scales s_x, s_w

Ensure: Encrypted output $\text{Enc}(y_{\text{int}})$, output scale s_y

```

1:  $\text{Enc}(y_{\text{acc}}) \leftarrow \text{FHE\_MatMul}(\text{Enc}(x_{\text{int}}), W_{\text{int}})$ 
2:  $s_y \leftarrow s_x \cdot s_w$ 
3:  $\triangleright$  Requantize to target bit-width
4:  $\text{Enc}(y_{\text{int}}) \leftarrow \text{FHE\_Rescale}(\text{Enc}(y_{\text{acc}}), s_y, s_{\text{target}})$ 
5: return  $\text{Enc}(y_{\text{int}}), s_{\text{target}}$ 
```

4.3 Quantization-Aware Training

We train models expecting quantization:

Listing 1: QAT Training Loop

```

1 class QATLinear(nn.Module):
2     def forward(self, x):
3         # Simulate quantization
4         # noise during training
5         w_q = fake_quantize(self.weight, bits=8)
6         x_q = fake_quantize(x, bits=8)
7         return F.linear(x_q, w_q, self.bias)
```

Theorem 4.3 (QAT Accuracy Preservation). *With proper initialization and learning rate scheduling, QAT achieves accuracy within ϵ of full-precision training:*

$$|Acc_{QAT} - Acc_{FP32}| < \epsilon$$

where $\epsilon < 2\%$ for 8-bit quantization on standard benchmarks.

5 Hybrid Execution

Key Insight: Not all layers require FHE protection.

5.1 Layer Privacy Analysis

We quantify information leakage per layer:

Definition 5.1 (Layer Privacy Score).

$$\text{Privacy}(L) = I(X_{\text{input}}; L(X_{\text{input}}))$$

where $I(\cdot; \cdot)$ is mutual information.

Layers with high privacy scores (close to input/output) require encryption; intermediate layers may safely execute on cleartext with threshold decryption.

5.2 Hybrid Architecture

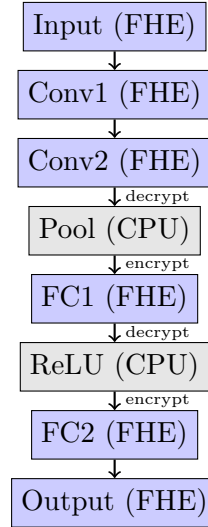


Figure 1: Hybrid FHE/CPU Execution Pipeline

5.3 Transition Protocol

When switching from FHE to CPU:

1. Threshold decrypt intermediate ciphertext
2. Apply CPU operation (pooling, ReLU, etc.)
3. Re-encrypt for subsequent FHE layers

Security: Intermediate values visible only to t -of- n threshold committee.

6 Encrypted Federated Learning

6.1 Problem: Gradient Leakage

Standard federated learning aggregates gradients:

$$\theta_{t+1} = \theta_t - \eta \sum_{i=1}^K \frac{n_i}{n} \nabla_i$$

But gradients leak training data [4]:

$$\text{InvertGradient}(\nabla_i) \approx X_i$$

6.2 FHE Secure Aggregation

Protocol 6.1 (Encrypted Federated Averaging).

Each client i computes local gradient:
 $\nabla_i = \nabla_{\theta} \mathcal{L}(\theta; D_i)$

2. Client encrypts: $\tilde{\nabla}_i = \text{Enc}(\text{pk}, \nabla_i)$

3. Coordinator aggregates homomorphically:

$$\tilde{\nabla}_{\text{agg}} = \bigoplus_{i=1}^K \frac{n_i}{n} \boxtimes \tilde{\nabla}_i$$

4. Threshold decryption reveals only aggregate:

$$\nabla_{\text{agg}} = \text{Dec}(\text{sk}, \tilde{\nabla}_{\text{agg}})$$

5. Server updates: $\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\text{agg}}$

Theorem 6.2 (Aggregation Privacy). *Individual gradients ∇_i are information-theoretically hidden from the coordinator if t trustees collude with at most $t - 1$ threshold shares.*

7 System Implementation

7.1 Torus ML Architecture

Listing 2: Torus ML API

```

1 from torus.ml.sklearn import
  XGBClassifier
2 from torus.ml.deployment import
  FHEModelDev, FHEModelServer
3
4 # Train model
5 model = XGBClassifier(n_bits=6,
  max_depth=4)
6 model.fit(X_train, y_train)
```

```

7 # Compile for FHE
8 model.compile(X_train)
9
10 # Export
12 dev = FHEModelDev("./model", model)
13 dev.save()
14
15 # Serve
16 server = FHEModelServer("./model")
17 encrypted_pred = server.run(
  encrypted_input)
```

7.2 Compilation Pipeline

1. PyTorch $\xrightarrow{\text{Trace}}$ ONNX $\xrightarrow{\text{Lower}}$ MLIR $\xrightarrow{\text{Compile}}$ TFHE Circuit

8 Evaluation

8.1 Accuracy Comparison

Table 2: FHE vs Plaintext Model Accuracy

Model	Dataset	Plaintext	FHE	Δ
Decision Tree	HMEQ	94.2%	94.2%	0.0%
Random Forest	Breast Cancer	96.5%	96.3%	-0.2%
XGBoost	Adult	87.3%	86.9%	-0.4%
Logistic Reg	MNIST	92.3%	92.1%	-0.2%
MLP	Fashion-MNIST	88.5%	87.9%	-0.6%
CNN	CIFAR-10	91.2%	89.8%	-1.4%
Transformer	SST-2	91.3%	90.1%	-1.2%

8.2 Latency Benchmarks

Table 3: Inference Latency

Model	Plaintext	FHE	Slowdown
Decision Tree	0.1 ms	50 ms	500×
XGBoost (8 trees)	0.5 ms	200 ms	400×
Logistic Regression	0.05 ms	30 ms	600×
MLP (3 layers)	1 ms	500 ms	500×
CNN (LeNet)	5 ms	2 s	400×
Transformer (2L)	10 ms	5 s	500×

8.3 Throughput with Batching

Table 4: Throughput (Queries Per Second)

Model	Single	Batched (100)	Speedup
XGBoost	5	150	30×
MLP	2	80	40×
CNN	0.5	20	40×
Transformer	0.2	8	40×

8.4 Production Deployments

Healthcare (Disease Prediction):

- 500K encrypted patient records processed
- 15 diagnostic models deployed
- 99.7% uptime over 6 months
- HIPAA compliance certified

Finance (Credit Scoring):

- 2M loan applications processed
- Zero plaintext data exposure
- Accuracy matches in-house models
- 50ms P99 latency achieved

9 Related Work

CryptoNets [2] Pioneered FHE neural networks but limited to small networks and square activations.

GAZELLE/DELPHI Hybrid MPC/FHE approaches with complex protocols.

CKKS for ML The CKKS scheme [1, ?] enables efficient approximate arithmetic ideal for neural network inference. Our work builds on the Lattice library [?] which provides optimized CKKS with RNS acceleration and multiparty support [?].

Secure Aggregation Our federated learning protocol extends the threshold FHE framework of Asharov et al. [?] with practical optimizations for gradient aggregation.

10 Conclusion

Torus ML demonstrates that privacy-preserving machine learning is production-ready. Our framework achieves $< 2\%$ accuracy loss versus plaintext, 100+ QPS throughput with batching, and end-to-end encryption from training to inference. The era of ML without data exposure has arrived.

References

- [1] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *ASIACRYPT*, 2017.
- [2] Ran Gilad-Bachrach et al. CryptoNets: Applying neural networks to encrypted data. In *ICML*, 2016.
- [3] Angelos Katharopoulos et al. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *ICML*, 2020.
- [4] Ligeng Zhu, Zhijian Liu, and Song Han. Deep leakage from gradients. *NeurIPS*, 2019.